KG 구축 K2 엔티티와 관계 추출

추출 단위 하나와 그 문서 유형의 그래프 스키마를 LLM에 넣어, 스키마 모양대로 노드와 관계를 JSON으로 받음

LLM에 넣는 것

① 문서 유형별 graph_schema.json
상담 기록: 노드 4, 관계 4와 해당, 비해당 설명

② 머리줄 (메타데이터 5칸)
record_id, 상담일, 채널, 주제, 가명 회원 키

③ 추출 단위 본문 (K1 결과)
상담은 검색용 청크 1개가 곧 단위 1개

④ 추출 지시 (라이브러리 기본)
JSON으로, 주어진 타입만, 관계 방향 지킴

→

LLM
gpt-oss-120b
(Groq)
단위 하나에 1번
동시 4건

→

나온 그래프 (v0 표본 실행, 상담 C-20260501-003)

Consultation C-20260501-003, 콜센터	**HAS_PROPOSAL**	**Proposal** 종이 영수증 대조 (거절)
Consultation C-20260501-003	**HAS_PROPOSAL**	**Proposal** 주문 내역 화면 대조 (수용)
Proposal 주문 내역 화면 대조	**RECONSIDER_WHEN**	**ReconsiderCondition** 다음 명세서 나올 때 다시 문의
Consultation C-20260501-003	**FOLLOWS**	**Consultation** C-20260302-002 (앞 상담)

1 같은 본문은 다시 묻지 않음
본문 해시와 스키마 지문이 같으면 앞 결과를 씀

2 형식 오류는 다시
같은 입력으로 2번까지 다시 물음

3 실패는 기록하고 계속
failed_units에 남기고 나머지 단위는 진행

4 노드 ID 앞 단위 ID는 임시
단위끼리 ID가 겹치지 않게 함. K4 적재 중에만 씀

v0 표본: 전체 재색인 단위 24개, LLM 24번, 실패 0, 토큰 76,401 / 증분 색인 LLM 1번, 3개 재사용
결과는 K3까지 마친 뒤 kg_prepared.json에 보관만 함. Neo4j에는 K4 반영 때 씀

무단전재 및 배포 금지

K2 추출 코드 (kg_build.py)

추출기는 한 번 만들고, 단위마다 머리줄을 붙여 부르며, K3 가지치기까지 마친 결과를 파일로 보관함

추출기 만들기와 _k2_k3_one (kg_build.py 발췌)

```
# prepare() 안: 추출기와 가지치기는 한 번만 만듦
llm = make_llm()                              # Groq openai/gpt-oss-120b
extractor = LLMEntityRelationExtractor(llm=llm,
    create_lexical_graph=False, on_error=OnError.RAISE)
pruner, sem = GraphPruning(), asyncio.Semaphore(config.LLM_CONCURRENCY)

async def _k2_k3_one(sem, extractor, pruner, unit, spec):
    async with sem:                           # 동시 4건
        text = _head_line(unit["metadata"], spec["header"]) + unit["text"]
        for attempt in range(config.LLM_RETRIES + 1):    # 2번까지 다시
            try:
                graph = await extractor.run(chunks=TextChunks(chunks=[
                    TextChunk(text=text, index=0, uid=unit["unit_id"])]),
                    schema=spec["schema"])
                break
            except Exception as e:
                if attempt == config.LLM_RETRIES:   # 실패는 기록용으로 돌려줌
                    return {"error": f"{type(e).__name__}: {str(e)[:200]}"}
        pruned = (await pruner.run(graph, spec["schema"])).graph   # K3
        return {"graph": pruned.model_dump(), ...}
```

kg_prepared.json에 보관한 결과 (일부)

```
{"unit_id": "D3_C-20260501-003_c797..._0000",
 "text_hash": "...", "schema_signature": "...",
 "graph": {"nodes": [ ...,
  {"id": "D3_C-20260501-003_c797..._0000:p1",
   "label": "Proposal",
   "properties": {"content": "그러면 주문 내역 화면으로 ...",
                  "decision": "수용"}}, ... ],
  "relationships": [ ...,
  {"type": "RECONSIDER_WHEN",
   "start_node_id": "..._0000:p1", "end_node_id": "..._0000:r0"}]}}
```

앞 세대 결과 재사용 (prepare 발췌)

```
hit = cache.get((u["doc_type"], u["text_hash"], u["schema_signature"]))
if hit:
    u.update(graph=hit["graph"], reused=True)   # LLM을 부르지 않음
else:
    todo.append(u)
```

응답 다듬기 (GroqLLM._clean)

정해진 칸 밖 키 제거, id는 글자로, 빈 속성 제거

무단전재 및 배포 금지